

My set-up consists of Fedora 13 (x86_64) and the latest *linux-next* kernel.

Setting up DebugFS

If you are using one of the latest distributions, chances are that DebugFS is already set up on your machine. If you're compiling the kernel from scratch, make sure you enable DebugFS in the kernel configuration. Once you reboot to your newly compiled kernel, check if DebugFS is already mounted, with the following command:

```
# mount | grep debugfs
none on /sys/kernel/debug type debugfs (rw)
```

If you see output as above, you have DebugFS pre-mounted. If not, you can mount it (as root) with the command shown below:

```
#mount -t debugfs nodev /sys/kernel/debug
```

If you want it to be available on every reboot, add an entry in */etc/fstab* as follows:

```
debugfs /sys/kernel/debug debugfs defaults 0 0
```

Once mounted, you can view a lot of files and directories in */sys/kernel/debug*, each belonging to one or the other subsystem.

The DebugFS API

To access the API, include *linux/debugfs.h* in your source file. To be able to use DebugFS, we start by creating a directory within */sys/kernel/debug*, which is an ideal way to start. The rest of the files can be placed within this directory.

```
struct dentry *debugfs_create_dir(const char *name, struct dentry *parent);
```

Here, *name* is the name of the directory, and *parent* is the parent directory (if null, the directory is created in */sys/kernel/debug*). If DebugFS is not enabled in the kernel, *ENODEV* is returned.

If you need to create a single file within DebugFS, you can call the following function:

```
struct dentry *debugfs_create_file(const char *name, mode_t mode, struct dentry *parent, void *data, struct file_operations *fops);
```

Here, *name* is the filename to be created; *mode* stands for the permissions of the created file; *parent* specifies the parent directory in which the file should be created (if null, it defaults to the DebugFS root, */sys/kernel/debug*); *data* is the type *inode.i_private* and *fops* is the file operations.

If you need to write to and read from a single value, you can use this to create an unsigned 8bit value:

```
struct dentry *debugfs_create_u8(const char *name, mode_t mode, struct dentry *parent, u8 *value);
```

Here, *value* is a pointer to a variable that needs to be read and written to.

A few other helper functions to create files with single integer values are:

```
struct dentry *debugfs_create_u16
struct dentry *debugfs_create_u32
struct dentry *debugfs_create_u64
```

(Refer to *fs/debugfs/file.c* for more information.)

Similar functions that give hex output, are:

```
dentry *debugfs_create_x8(const char *name, mode_t mode, struct dentry *parent, u8 *value)
dentry *debugfs_create_x16
dentry *debugfs_create_x32
^Adentry *debugfs_create_x64
```



Note: *debugfs_create_x64* is the most recent edition of the API; added in May 2010 by Huang Ying, this tells us that DebugFS is still in active development.

Clean up!

One important thing to remember is that all the files and directories created using the above APIs are to be cleaned up by the creator, using the following code:

```
void debugfs_remove(struct dentry *dentry)
```

Here, *dentry* is a pointer to the file or directory that needs to be cleaned up.

Other available DebugFS APIs are:

```
debugfs_create_symlink - create symbolic link in DebugFS
debugfs_remove_recursive - recursive removal of a directory tree
debugfs_rename - rename a file or directory
debugfs_initialized - to know whether DebugFS is registered
debugfs_create_bool - to read/write a boolean value
debugfs_create_blob - to read a binary large object
```

You can refer to code for these and other functions in *fs/debugfs/{inode.c, file.c}* in the kernel directory.

On a general note, DebugFS is not considered to be a stable Application Binary Interface (ABI), and hence it should only be used to collect debug information on a temporary basis. For more information in this regard, you can refer to a *Linux Weekly News* article, at: <http://lwn.net/Articles/309298/>.